
University of Kansas

Arithmetic Expression Evaluator C++

Test Case

Version 1.3

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

Revision History

Date	Version	Description	Author
01/May/26	1.0	Added Test Cases for Parser	Yecolia E.
05/May/26	1.1	Added test cases for Evaluator module	John P.
07/May/26	1.2	Added tokenizer and system integration test cases	Yecolia E.
07/May/26	1.3	Added general test cases, more detail to purpose section, and updated table of contents.	John P.

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

Table of Contents

1. Purpose	4
1.1 General test cases (TC)	4
1.2 Evaluator test cases (EV-TC)	4
1.3 Parser test cases (P-TC)	4
1.4 Tokenizer test cases (T-TC)	4
1.5 System test cases (SYS-TC)	4
2. Test case table	4

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

Test Case

1. Purpose

This Test Case Specification for the Arithmetic Expression Evaluator defines the test cases used to verify the software system's correctness, reliability, and functionality. The document includes test cases for the tokenizer, parser, evaluator, error handling, and supported arithmetic operations to ensure the software behaves as expected under valid and invalid inputs. The tests were divided into multiple categories.

1.1 General test cases (TC)

These test cases evaluate the overall behavior of the evaluator from the user's point of view. They focus on complete expressions, operator precedence, parentheses handling, unary operators, and error detection.

1.2 Evaluator test cases (EV-TC)

These test cases target the evaluation stage of the program. They verify that the evaluator completes the correct result given an input of a prefix notation token stream. These tests focus on arithmetic correctness and error handling in the evaluation process.

1.3 Parser test cases (P-TC)

These test cases target the parsing stage of the program. They ensure that invalid syntax, unmatched parentheses, missing operands, and other errors are handled. Also, these tests confirm associativity, and precedence concerns are handled.

1.4 Tokenizer test cases (T-TC)

These test cases target the tokenizer stage of the evaluation stage of the program. They ensure that the tokenizer can recognize different tokens, such as parentheses and operators. Also, these tests confirm the user can safely enter valid expressions, such as ignoring extra white space one might enter.

1.5 System test cases (SYS-TC)

These test cases validate interactions among the tokenizer, parser, and evaluator modules. They ensure that basic valid and invalid inputs are handled involving the included operators.

2. Test case table

Test Case ID	Description	Test Data	Expected Results	Actual Results	Pass/Fail Status
TC01	Verifies calculator correctly performs simple addition of two integers	$3 + 4$	7	7	Pass
TC02	Ensures expressions inside parentheses are evaluated before subtraction	$8 - (5 - 2)$	5	5	Pass
TC03	Verifies multiplication	$10 * 2 / 5$	4	4	Pass

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

	and division follow left-to-right associativity				
TC04	Confirms exponent operator (**) works correctly	$2 ** 3$	8	8	Pass
TC05	Tests combination of *, +, %, and - with correct precedence	$4 * (3 + 2) \% 7 - 1$	5	5	Pass
TC06	Multiple nested parentheses that do not affect correctness	$((2 + 3))) + (((1 + 2)))$	8	8	Pass
TC07	Mixed operators with redundant parentheses	$((5 * 2) - ((3 / 1) + ((4 \% 3))))$	6	6	Pass
TC08	Nested parentheses with exponentiation and division	$((2 ** (1 + 1)) + ((3 - 1) ** 2)) / ((4 / 2) \% 3)$	4	4	Pass
TC09	Extraneous and necessary parentheses combined in arithmetic expression	$(((((5 - 3))) * (((2 + 1))) + ((2 * 3))))$	12	12	Pass
TC10	Division with multiple nested parentheses	$((9 + 6)) / ((3 * 1) / (((2 + 2))) - 1)$	-60	-60	Pass
TC11	Unary operators with multiplication, division, and subtraction	$+(-2) * (-3) - ((-4) / (+5))$	6.8	6.8	Pass
TC12	Unary negation inside parentheses followed by addition	$-(+1) + (+2)$	1	1	Pass
TC13	Nested unary negation with	$-(-(-3)) + (-4) + (+5)$	-2	-2	Pass

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

	addition and subtraction				
TC14	Unary operators with exponentiation producing fractional result	$+2 ** (-3)$	0.125	0.125	Pass
TC15	Combination of unary operators with parentheses and arithmetic operations	$-(+2) * (+3) - (-4) / (-5)$	-6.8	-6.8	Pass
TC16	Unmatched parentheses causing syntax error	$2 * (4 + 3 - 1$	"Error: Parser error at index 8: Expected closing parenthesis ')'"	"Error: Parser error at index 8: Expected closing parenthesis ')'"	Pass
TC17	Operator without left operand	$* 5 + 2$	"Error: Parser error at index 0: Unexpected token: '"	"Error: Parser error at index 0: Unexpected token: '"	Pass
TC18	Division by zero (mathematically undefined)	$4 / 0$	"Evaluation Error [Code 1]: Division or Modulo by zero is undefined."	"Evaluation Error [Code 1]: Division or Modulo by zero is undefined."	Pass
TC19	Missing operator between operand and parentheses	$5 (2 + 3)$	"Error: Parser error at index 1: Unexpected token after end of expression: ('"	"Error: Parser error at index 1: Unexpected token after end of expression: ('"	Pass
TC20	Invalid character not supported by grammar	$7 \& 3$	"Error: Tokenizer error at index 1"	"Error: Tokenizer error at index 1"	Pass
TC21	Mismatched parentheses with missing closing bracket	$((3 + 4) - 2) + (1$	"Error: Parser error at index 14: Expected closing parenthesis ')'"	"Error: Parser error at index 14: Expected closing parenthesis ')'"	Pass
TC22	Division by zero inside complex expression	$((5 + 2) / (3 * 0))$	"Evaluation Error [Code 1]: Division or Modulo by zero is undefined."	"Evaluation Error [Code 1]: Division or Modulo by	Pass

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

				zero is undefined.”	
TC23	Invalid operator sequence (operator missing operand)	$((2 -) 1 + 3)$	“Error: Parser error at index 4: Unexpected token:)”	“Error: Parser error at index 4: Unexpected token:)”	Pass
TC24	Missing operand after operator	$((4 * 2) + (-))$	“Error: Parser error at index 9: Missing operand after unary ‘_’”	“Error: Parser error at index 9: Missing operand after unary ‘_’”	Pass
TC25	Invalid exponent operator not supported by grammar	$((7 * 3) ^ 2)$	“Error: Tokenizer error at index 6”	“Error: Tokenizer error at index 6”	Pass
TC26	Tests right-to-left associativity for exponentiation	$2 ** 3 ** 2$	512	512	Pass
TC27	Verifies the evaluator can handle large integer arithmetic	$12345 + 67890$	80235	80235	Pass
TC28	Tests modulo inside a multi-operator expression	$25 \% 7 + 3 * 2$	10	10	Pass
TC29	Confirms operator precedence rules are followed	$25 + 5 * 4 / 2 - 3$	32	32	Pass
TC30	Ensures evaluator handles expression if tokens separated by arbitrary number of space characters	$5 \quad * 2$	10	10	Pass
TC31	Verifies nested parentheses with modulo	$((25 \% (3 + 2)) * (6 - 4))$	0	0	Pass

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

	are handled properly				
TC32	Confirms the evaluator handles complex integer expression mixing	$(2 ** 3) * (9 \% 4) - (6 - 3)$	5	5	Pass
TC33	Ensures repeated unary signs are resolved	---5	-5	-5	Pass
TC34	Confirms unary minus works with exponentiation	$-2 ** 3$	-8	-8	Pass
TC35	Tests unary minus in deep nesting of parentheses	$((-(3)) + (-(2)))$	-5	-5	Pass
TC36	Ensures unary plus and minus behave properly	$+2 - -3 + -4$	1	1	Pass
TC37	Confirm modulo works with negative numbers	$-15 \% 4$	1	1	Pass
TC38	Tests that the optional "history" feature successfully returns previous entered expressions	1. Enter "3 + 4" 2. Enter "5 * 7" 3. Enter "history"	Expression History: 1.) 3 + 4 2.) 5 * 7	Expression History: 1.) 3 + 4 2.) 5 * 7	Pass
EV-TC01	Confirms the evaluator can handle basic binary arithmetic with multiplication and addition	Prefix: $+ 3 * 4 2$	11	11	Pass
EV-TC02	Ensures the evaluator can handle basic binary arithmetic with	Prefix: $- 10 / 6 3$	8	8	Pass

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

	division and subtraction				
EV-TC03	Validates the modular arithmetic with subtraction	Prefix: - 10 % 7 3	9	9	Pass
EV-TC04	Confirms the evaluator can handle exponentiation with nested exponent	Prefix: ** 2 ** 2 3	256	256	Pass
EV-TC05	Validates the evaluator can handle the unary minus within an evaluation	Prefix: - * 5 3	-15	-15	Pass
EV-TC06	Confirms the evaluator could possibly detect invalid tokens	Prefix: + 4 * 3 # 2 5	3 or UNKNOWN_TOKEN	3	Pass
EV-TC07	Ensures the evaluator can detect division by zero (invalid)	Prefix: / 20 0	1 or DIV_ZERO	1	Pass
EV-TC08	Confirm consistent evaluation when operators occur across recursion layers	Prefix: + * - 20 5 3 / ** 8 2 2	77	77	Pass
P-TC01	Verify parser handles operator precedence (multiplication before addition)	3 + 4 * 2	Prefix: + 3 * 4 2	+ 3 * 4 2	Pass
P-TC02	Verify parser handles parentheses overriding precedence	(3 + 4) * 2	Prefix: * + 3 4 2	* + 3 4 2	Pass
P-TC03	Verify parser handles nested expressions	8 - (5 - 2)	Prefix: - 8 - 5 2	- 8 - 5 2	Pass

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

P-TC04	Verify parser handles left-associative subtraction	10 - 3 - 2	Prefix: - - 10 3 2	- - 10 3 2	Pass
P-TC05	Verify parser handles right-associative exponentiation	2 ** 3 ** 2	Prefix: ** 2 ** 3 2	** 2 ** 3 2	Pass
P-TC06	Verify parser handles single number input	42	Prefix: 42	42	Pass
P-TC07	Verify parser handles deeply nested parentheses	((2 + 3) * (4 - 1))	Prefix: * + 2 3 - 4 1	* + 2 3 - 4 1	Pass
P-TC08	Detect missing right operand	3 +	Error: Unexpected end of expression	Unexpected end of expression	Pass
P-TC09	Detect missing closing parenthesis	(3 + 4	Error: Expected closing parenthesis	Expected closing parenthesis ')'	Pass
P-TC10	Detect extra closing parenthesis	3 + 4)	Error: Unexpected token after end of expression	Unexpected token after end of expression:)	Pass
T-TC01	Verify tokenizer separates basic numbers and operators	3 + 4 * 2	Tokens: 3 + 4 * 2	Tokens: 3 + 4 * 2	Pass
T-TC02	Verify tokenizer ignores extra spaces	" 12 + 7 " with extra spaces	Tokens: 12 + 7	Tokens: 12 + 7	Pass
T-TC03	Verify tokenizer recognizes parentheses	(3 + 4) * 2	Tokens: (3 + 4) * 2	Tokens: (3 + 4) * 2	Pass
T-TC04	Verify tokenizer recognizes exponent operator	2 ** 3	Tokens: 2 ** 3	Tokens: 2 ** 3	Pass
T-TC05	Verify tokenizer handles chained	10 - 3 - 2	Tokens: 10 - 3 - 2	Tokens: 10 - 3 - 2	Pass

Arithmetic Expression Evaluator C++	Version: 1.3
Test Case	Date: 07/May/26
EECS348-TC-TermProj-v1.3	

	subtraction				
T-TC06	Verify tokenizer recognizes modulo operator	$7 \% 2$	Tokens: $7 \% 2$	Tokens: $7 \% 2$	Pass
T-TC07	Verify tokenizer handles subtraction and parentheses	$8 - (5 - 2)$	Tokens: $8 - (5 - 2)$	Tokens: $8 - (5 - 2)$	Pass
T-TC08	Verify tokenizer handles mixed operators and parentheses	$3 + 4 * 2 / (1 - 5)$	Tokens: $3 + 4 * 2 / (1 - 5)$	Tokens: $3 + 4 * 2 / (1 - 5)$	Pass
SYS-TC01	Verify complete evaluation of basic expression	$3 + 4 * 2$	11	11	Pass
SYS-TC02	Verify parentheses precedence	$(3 + 4) * 2$	14	14	Pass
SYS-TC03	Verify exponentiation associativity	$2 ** 3 ** 2$	512	512	Pass
SYS-TC04	Verify unary negation	$-(2 + 3) * 4$	-20	-20	Pass
SYS-TC05	Verify invalid expression handling	$3 +$	Parser error	Parser error: Unexpected end of expression	Pass